Introducción a la Teoría de Autómatas y LF

Lenguajes Formales y de Programación
Sección A-

Inga. Damaris Campos

¿Por qué estudiamos Teoría de Autómatas y Lenguajes Formales?

Los lenguajes formales y los autómatas son el **fundamento de casi todo** lo que hacemos en programación: desde entender cómo se procesan las instrucciones de un lenguaje de alto nivel hasta cómo funcionan los compiladores y sistemas operativos.

¿Qué aprenderemos?

- Aprenderás cómo los lenguajes de programación están diseñados.
- Cómo los entienden las máquinas.
- Herramientas poderosas como autómatas finitos, expresiones regulares y gramáticas, que son la base de la informática moderna.

¿Para qué nos prepara?

Este curso es un puente hacia áreas más complejas como la inteligencia artificial, el desarrollo de algoritmos eficientes y el diseño de sistemas críticos.

Lenguaje Natural – Lenguaje Formal

Desarrollado por el ser humano, como medio de comunicación cotidiana con los demás.

- Se desarrolla por enriquecimiento progresivo, antes de cualquier formalización completa.
- Dificultad o imposibilidad de formalización completa.
- Polisemántica, puede ser ambiguo o interpretarse dependiendo del contexto.
- Poder EXPRESIVO.

Un lenguaje desarrollado por el hombre, para expresar situaciones específicas en un área de conocimiento.

- Se desarrolla de una teoría preestablecida – ESTRUCTURA.
- Requiere completa formalización, para implementarlo en una construcción computacional.
- La sintaxis debe producir sentencias NO AMBIGUAS.
- Componente semántico MÍNIMO.
- La importancia del rol de los números.
- Poder expresivo LIMITADO!

¿Qué es un Lenguaje de Programación y cómo se clasifican?

Los lenguajes de programación son lenguajes FORMALES que sirven para escribir programas que permitan la comunicación usuario – máquina. (Joyanes; 2003)

Según paradigma de programación:

Imperativos

Se ordena a la computadora cómo realizar una tarea, siguiendo una serie de pasos o instrucciones.

Declarativos

Se le indica a la computadora qué es lo que se desea obtener o qué es lo que se está buscando (Lógicos y Funcionales).

Según forma de ejecución:

Compilados

Aquellos que se ejecutan como un todo, pasando por uno o varios procesos de traducción hasta ser comprensibles por la computadora.

Interpretados

Aquellos que se ejecutan instrucción por instrucción, haciendo traducción de un determinado lenguaje a un idioma comprensible para la computadora.

Según su nivel de abstracción:

Alto Nivel

Lenguajes independientes de la arquitectura del computador, por lo que se puede migrar de una máquina a otra tomando en cuenta únicamente los factores de compatibilidad.

Mediano Nivel

Tienen características que los acercan a los lenguajes de bajo nivel, pero, al mismo tiempo, ciertas cualidades que lo hacen un lenguaje más cercano al humano y, por tanto, de alto nivel.

Bajo Nivel

Son lenguajes dependientes de la arquitectura del computador, no se puede migrar o utilizar en otras máquinas sin recodificación.

Los lenguajes más cercanos a la arquitectura hardware se denominan lenguajes de bajo nivel, y los que se encuentran más cercanos a los programadores y usuarios se denominan lenguajes de alto nivel.

Lenguaje de bajo Nivel:

Lenguaje de Máquina

Es el que da órdenes a la máquina, que son las operaciones fundamentales para su funcionamiento, en código binario o código máquina, consistente en ceros y unos.

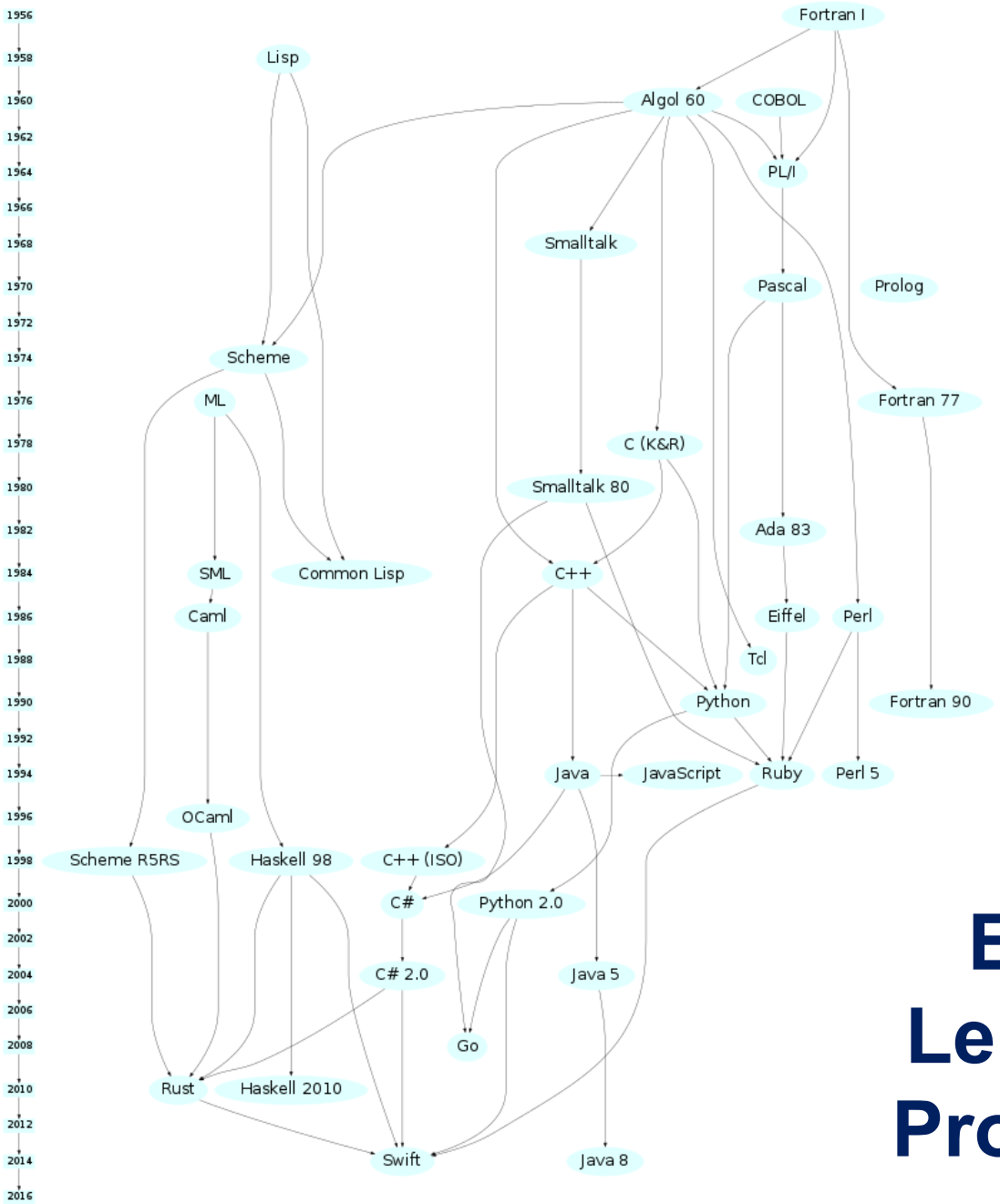
Lenguaje Ensamblador

Es un derivado del lenguaje máquina y está formado por abreviaturas de letras y números.

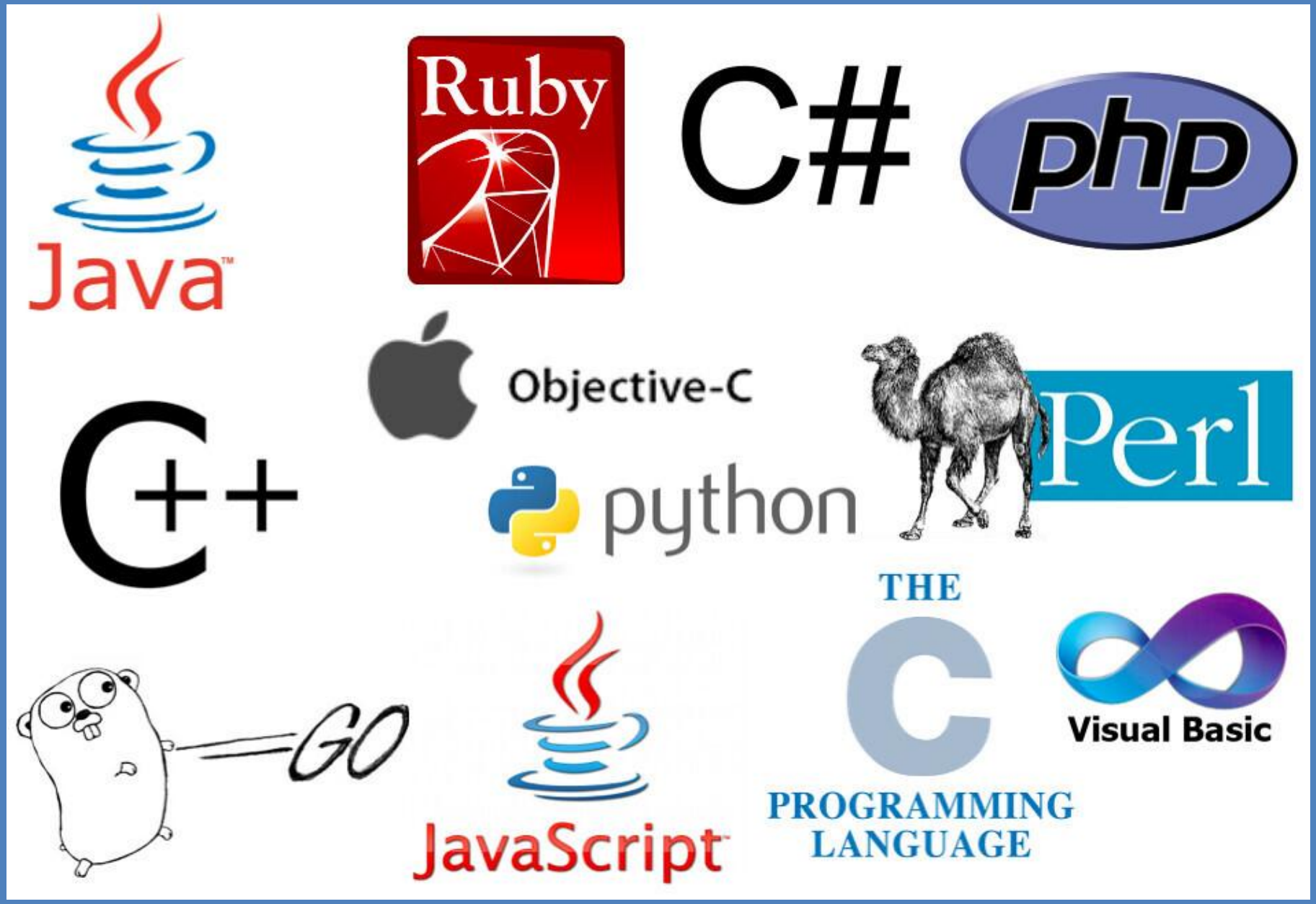
Se trabaja a nivel de instrucciones, es decir, su programación es al más fino detalle.

First programming language was introduced in 1956

Programmers before 1956 :



Evolución Lenguajes de Programación


Java
Ruby
C#
php
Objective-C
C++
python
Perl
THE
C
PROGRAMMING
LANGUAGE
Visual Basic
GO
JavaScript

Programming Languages You Should Learn To Become

Web Developer

Game Developer

Data Analysis

Desktop Developer

Embedded System program

Mobile Apps Development

TIOBE Enero 2026 TOP 10

Lenguajes de Programación más usados

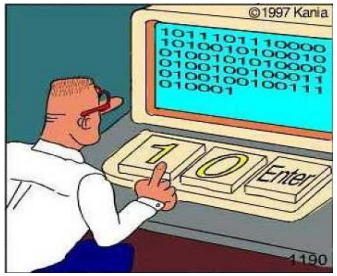
Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	^	C	10.99%	+2.13%
3	3		Java	8.71%	-1.44%
4	2	v	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	^	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	^	Delphi/Object Pascal	1.98%	+0.19%
10	18	^^	R	1.82%	+0.81%
11	32	^^	Perl	1.63%	+1.14%
12	10	v	Fortran	1.61%	-0.42%
13	14	^	Rust	1.51%	+0.34%
14	15	^	MATLAB	1.40%	+0.34%
15	13	v	PHP	1.38%	-0.00%
16	7	vv	Go	1.24%	-1.37%
17	12	vv	Scratch	1.24%	-0.31%
18	26	^^	Ada	1.19%	+0.54%
19	17	v	Assembly language	1.07%	+0.05%
20	25	^^	Kotlin	0.97%	+0.23%

Por generaciones:

Primera
Lenguaje
de
Máquina

Segunda
Lenguajes
Ensamblador
es

Tercera
Lenguajes
de Alto
Nivel
Funcionale
s

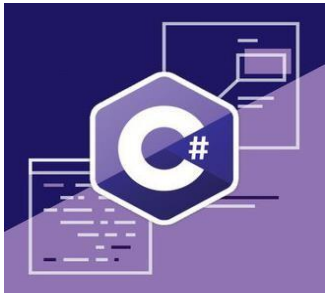


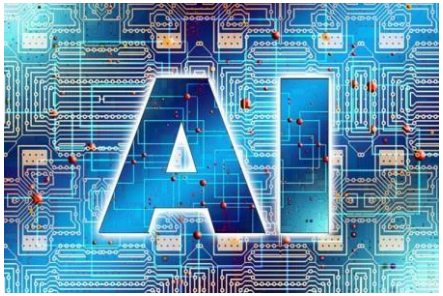
Por generaciones:

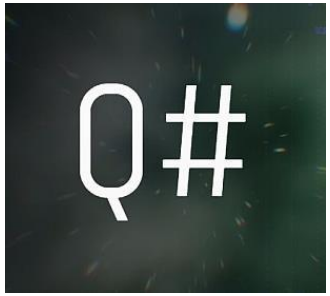
Cuarta
Lenguajes de Alto
Nivel
Parametrizables

Quinta
Lenguajes orientados a la Inteligencia Artificial

Sexta
Lenguajes orientados a la computación cuántica







PRIMERA GENERACIÓN

(1940 - 1950)

- Sucesión Dígitos Binarios (0,1), conocidos como bits
- Único lenguaje interno que la computadora entiende
- Puesto que el hardware se desarrollaba antes que el software, estos lenguajes estaban adaptados a una máquina específica.

SEGUNDA GENERACIÓN

(1950 - 1955)

- Lenguajes Ensambladores
- Instrucciones Símbolos Mnemotécnicos

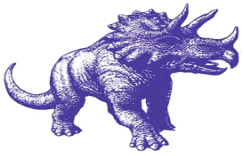
(Ejemplo: DIV, por dividir)

- Únicos para computadora en particular
- Son de bajo nivel por depender de la computadora
- Relación uno a uno entre instrucciones fuente, e instrucciones objeto

TERCERA GENERACIÓN

(años '60s)

- Los intérpretes admiten instrucciones más completas y se crean lenguajes con sus correspondientes frases y palabras.
- El primero lenguaje de programación de alto nivel es FORTRAN
- Hacen énfasis los procedimientos o las matemáticas implícitas, es decir en lo que se hace (la acción).
- Están diseñados para resolver un conjunto particular de problemas



COBOL

CUARTA GENERACIÓN

(Años ‘70s a ‘80s)

- Propósitos específicos
- Creación de prototipos de una aplicación
- Permiten la retroalimentación en estructura y diseño
- Capacidad para base de datos
- También aparecen los lenguajes de programación orientada a objetos

QUINTA GENERACIÓN

(Década de los ‘90s)

- Inteligencia artificial y sistemas de expertos
- Anticiparse a las necesidades del usuario
- Reconocer patrones y secuencias de procesamiento del lenguaje humano
- Mayor velocidad
- Difíciles de desarrollar

SWI Prolog

SEXTA GENERACIÓN

(Desde los años 2000)

- Permiten describir estados cuánticos, puertas lógicas cuánticas y circuitos complejos necesarios para resolver problemas
- Proveen herramientas para ejecutar simulaciones en hardware tradicional antes de usar computadoras cuánticas reales.

Cirq

Qiskit

¿Consultas?

damaris.campos@ingenieria.usac.edu.gt